EC991 Topics in Macroeconomics

Week 3 Tutorial · Solving Aiyagari, Part 1

Aseem Patel

Department of Economics, University of Essex

Baseline Aiyagari model

We now take the model to the computer. The ingredients, in the notation of the code:

- Aggregate production function: $Y_t = K_t^{\alpha} N_t^{1-\alpha}$
- Household problem, with the **log** endowment following an AR(1):

$$V(a,y) \;=\; \max_{a' \geq -\phi} \Big\{ u\big(Ra + w\,y - a'\big) + \beta \sum_{y'} P(y' \mid y)\, V(a', y') \Big\}, \qquad \log y_t = \rho \log y_{t-1} + \sigma \varepsilon_t, \quad \varepsilon \sim N(0,1)$$

  the AR(1) replaces week 2's finite chain; discretising it (below) gives back a grid $y_1, \ldots, y_{n_y}$ and week 2's transition probabilities $P(y' \mid y)$ as a matrix P

- We must solve for the interest rate that **clears the asset market**:

$$\int_{A \times Y} a \; d\Lambda(a,y) \;=\; K$$

  where Λ is the stationary distribution and K is capital demand from the firm's problem

Algorithm in a nutshell

The solution is characterised by a **fixed point** in the interest rate:

Step 1. Guess an interest rate r_0 (or a quantity of capital K_0)

Step 2. Use r_0 in the firm's first-order conditions to recover capital demand and the wage:

$$K(r_0) \;=\; \left(\frac{\alpha}{r_0+\delta}\right)^{\frac{1}{1-\alpha}} N, \qquad w(r_0) \;=\; (1-\alpha)\left(\frac{K(r_0)}{N}\right)^{\alpha}$$

Step 3. Given w and r_0, solve the household problem for the policy function $g_a(a, y;\, r_0)$:

$$V(a,y) \;=\; \max_{a' \geq -\phi} \left\{ u\big(Ra + wy - a'\big) \;+\; \beta \sum_{y'} P(y' \mid y)\, V(a', y') \right\}$$

Step 4. Given g_a, find the **stationary distribution**, the Λ that replicates itself under the transition function Q:

$$\Lambda(\mathcal{A} \times \mathcal{Y}) \;=\; \int_{A \times Y} Q\big((a,y),\, \mathcal{A} \times \mathcal{Y}\big)\, d\Lambda(a,y)$$

Step 5. Given Λ, compute **excess demand**, asset supply minus capital demand:

$$\Phi(r_0) \;=\; \int_{A \times Y} a \; d\Lambda(a, y;\, r_0) \;-\; K(r_0)$$

If $\Phi(r_0) = 0$: equilibrium found. Otherwise update the guess r_0 and return to step 2

Today: step 3, the household problem

- Of the five steps, today is about **step 3**: given prices (w, r), solve the household problem for the policy function $g_a(a, y;\ r)$
  - steps 1, 2 and 5 are short calculations once step 3 and step 4 are done
  - step 4, the stationary distribution, comes next week

- A computer cannot store a function on a continuum, so we need three pieces:

1.	**Discretise the asset space**	a grid $a_1, \ldots, a_{n_a}$: how many points, where to put them
2.	**Discretise the income process**	turn the AR(1) for $\log y$ into a grid $y_1, \ldots, y_{n_y}$ and a matrix P (Tauchen, Rouwenhorst)
3.	**Solve the Bellman equation on the grid**	value function iteration: $\underbrace{\text{grid search}}_{\text{week 2}}$, $\underbrace{\text{interpolation}}_{\text{this week}}$, $\underbrace{\text{speed-ups}}_{\text{next week}}$

- The output is the policy $g_a(a, y;\ r)$ at every grid point: the input to step 4

Discretisation

- We have to solve the household problem using **global methods**
  - global: solve on a grid over the **whole state space**, rather than a Taylor expansion around a steady state (perturbation, as in Dynare)
  - a local approach fails here: the borrowing constraint is a **kink**, first-order perturbation removes **precautionary saving**, and households are spread from the constraint to high wealth
- The first step is to **discretise the state space**
- Trade-off between **speed** and **accuracy**: more grid points give a more accurate solution, but the problem takes longer to solve
- You must always check that the solution is **not affected** by your grid choices
  - sanity check: increase the number of grid points and move the boundaries of the state space, and verify that nothing changes

Discretisation: assets

- Choose the number of points n_a, the bounds (a_1, a_{n_a}) and a spacing rule, so the grid is $g_A = [a_1, a_2, \ldots, a_{n_a}]'$
- **Bounds**: $a_1 = -\phi$; the upper bound a_{n_a} must be chosen so it **never binds**
  - the lower bound is a **real** constraint: households cannot go below $-\phi$, and the kink there is an economic feature to capture
  - the upper bound exists only because a computer needs a finite grid; nothing in the model stops a household from saving more
  - if it binds, the richest households are forced to save less than they want; the error spreads through the value function, and mass **piles up** at the top grid point, biasing aggregate capital and r
  - simple trick: take the steady-state k of the neoclassical growth model, roughly average wealth without income risk, and multiply by a constant (often 3 to 10).

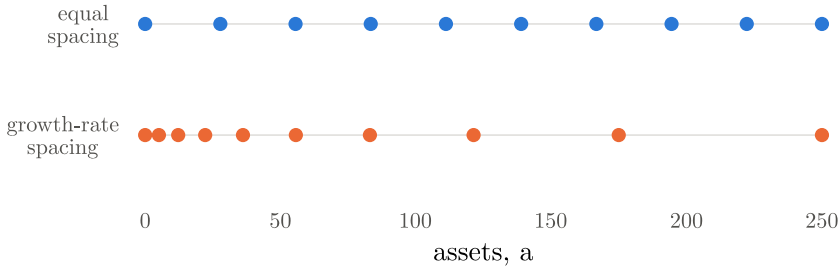
Discretisation: assets

- **Spacing**:
  - simplest: equal distance between points
  - alternative: more points where there is more curvature or kinks
  - why: the computer only knows the function at the grid points. If the function bends sharply between two points, a wide gap can **hide that bend**
  - in Aiyagari, the policy bends near the borrowing constraint, so put more points there:

$$a_i \;=\; a_1 + (a_{n_a} - a_1)\,\frac{(1+\nu)^{\,i-1} - 1}{(1+\nu)^{\,n_a-1} - 1}, \qquad i = 1, \ldots, n_a$$

Geometric grid spacing: successive gaps grow at rate ν, $a_{i+1} - a_i = (1+\nu)(a_i - a_{i-1})$. With $\nu = 0.4$, each gap is 40% larger than the previous one, so points cluster near the lower bound

Discretisation: assets



- Ten points on $[a_1, a_{n_a}] = [0, 250]$. With growth rate $\nu = 0.4$ the first gap is about 5 and the last about 75
- The growth-rate grid puts half its points below $a = 40$, where the borrowing constraint bends the policy function

Discretisation: assets

- People very often use the **same** grid to approximate the policy functions and the stationary distribution, but it does **NOT** have to be the case
  - the grids do two different jobs: what households **choose** (the policy) and where households **are** (the distribution)
  - a smooth policy can be solved on a coarse grid, say **100** asset points, and interpolated in between
  - the distribution may need a finer grid, say **1,000** points, to show where households are concentrated
  - at the extra points, interpolate the policy already computed instead of solving the maximisation again: a finer distribution at little extra cost
- The right number of points depends on the method used for the policy function and for the distribution:
  - less accurate methods require more grid points
  - if you plan to use interpolation, be extra careful
- Another strategy is a **multigrid** method: solve on a coarse grid, then use that solution as the guess on a finer grid
- Advanced references: Maliar and Maliar (2014, Handbook of Computational Economics); Brumm and Scheidegger (2017, Econometrica)

Discretisation: Markov chain

- We must discretise $\log y_t = \rho \log y_{t-1} + \sigma\varepsilon_t$, $\varepsilon \sim N(0,1)$ into a Markov chain with n_y points
- The output is a grid and a transition matrix:

$$\left[\log y_1,\ \log y_2,\ \ldots,\ \log y_{n_y}\right]', \qquad P = \begin{pmatrix} P_{11} & P_{12} & \ldots \\ \vdots & \ddots & \\ P_{n_y 1} & & P_{n_y n_y} \end{pmatrix}$$

  where the element P_{jk} of the $n_y \times n_y$ matrix is the probability of going from state j to state k:
  $P_{jk} = P(y_k \mid y_j) = \Pr(y' = y_k \mid y = y_j)$

- Two standard methods to discretise the AR(1): **Tauchen** and **Rouwenhorst**

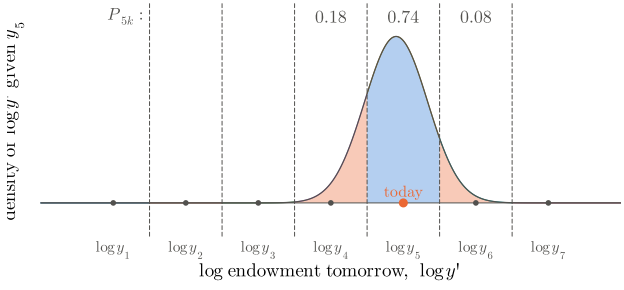
Discretisation: Tauchen

- References: Tauchen (1986) and Flodén (2008, Economics Letters)
- Start by choosing (symmetric) end points using the **unconditional** standard deviation:

$$-\log y_1 \;=\; \log y_{n_y} \;=\; m\left(\frac{\sigma^2}{1-\rho^2}\right)^{\frac{1}{2}}, \qquad m > 0$$

- Why the **unconditional** standard deviation:
  - σ measures only the size of today's shock; $\sigma/\sqrt{1-\rho^2}$ measures the spread of **income itself**, which carries the lingering effects of past shocks
  - with persistent shocks, a run of good or bad draws leaves income far from its mean: the grid must cover those levels
  - end points about three unconditional standard deviations either side of the mean cover most of the stationary income distribution
- Tauchen uses $m = 3$; Flodén advocates $m = 1.2\ln(n_y)$
- Choose **equidistant** points between $\log y_1$ and $\log y_{n_y}$; denote d the distance between points
  - another option is Gaussian nodes (Tauchen and Hussey, 1991)

Tauchen: the idea



$n_y = 7,\ \rho = 0.9,\ \sigma = 0.1$

Figure after Alisdair McKay's notes

- **Fix today's state**, say y_5. The AR(1) gives $\log y' = \rho \log y_5 + \sigma\varepsilon$: the bell curve, centred at $\rho \log y_5$
- **Each grid point stands for an interval**, bounded halfway to its neighbours (the dashed lines). If $\log y'$ lands in the interval around $\log y_k$, we record state y_k
- **The area is the probability**: the area inside the interval around $\log y_5$ is about 0.74, so $P_{55} \approx 0.74$. Likewise $P_{54} \approx 0.18$ and $P_{56} \approx 0.08$
- That is **row 5** of the transition matrix. Repeat from every starting state to fill the other rows

Tauchen: computing the areas

- For an interior destination y_k, the interval runs from $L = \log y_k - d/2$ to $U = \log y_k + d/2$, and we want $P_{jk} = \Pr\left(L \leq \log y' \leq U \,\middle|\, y_j\right)$
- A CDF gives the probability **to the left** of a point, so: probability inside the interval = probability below U − probability below L
- Given y_j, $\log y'$ has mean $\rho \log y_j$ and standard deviation σ. With Φ the CDF of the standard normal:

$$P_{jk} = \begin{cases} \Phi\left(\dfrac{\log y_1 - \rho \log y_j + d/2}{\sigma}\right) & \text{if } k = 1 \\ \Phi\left(\dfrac{\log y_k - \rho \log y_j + d/2}{\sigma}\right) - \Phi\left(\dfrac{\log y_k - \rho \log y_j - d/2}{\sigma}\right) & \text{if } 1 < k < n_y \\ 1 - \Phi\left(\dfrac{\log y_{n_y} - \rho \log y_j - d/2}{\sigma}\right) & \text{if } k = n_y \end{cases}$$

- The first and last cases collect **all remaining probability** in the left and right tails

Discretisation: Rouwenhorst

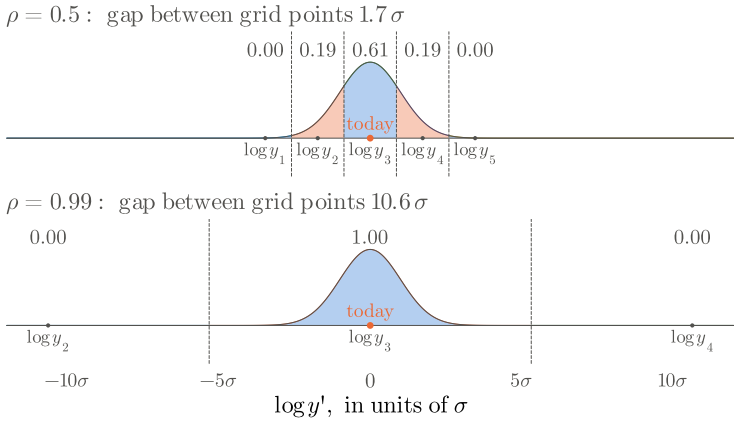
- **Why another method?** Tauchen's grid width scales with the **unconditional** spread $\sigma/\sqrt{1-\rho^2}$, which grows as $\rho \to 1$, while the one-step **conditional** spread stays at σ
  - with few points, the whole conditional bell curve falls inside one or two bins: the chain barely moves, and persistence, variance and autocorrelation are all off
  - the fix is more bins, until the gaps are small relative to σ: a persistent process needs **many points**, which is costly

- **Rouwenhorst sidesteps this**: for any $n_y \geq 2$, the chain matches the mean, variance and first-order autocorrelation of $\log y$ **exactly**
  - it never integrates a bell curve: it pins down algebraically the quantities Tauchen gets wrong at high ρ
  - its accuracy does not deteriorate as ρ rises, even near a unit root, so a **small chain** is enough

- The standard choice for $\rho > 0.9$; the main alternative is Farmer and Toda (2017, QE)

Kopecky and Suen 2010

Why another method? Tauchen at high persistence



Tauchen with $n_y = 5$, $\sigma = 0.1$, today at $\log y_3 = 0$; both rows on one axis in units of σ; numbers above the bins are P_{3k}; at $\rho = 0.99$, $\log y_1$ and $\log y_5$ sit at $\mp 21\sigma$, beyond the axis

- **Same bell curve in both panels**: the one-step spread is σ, whatever ρ
- **Different bins**: the grid spans $\pm 3\,\sigma/\sqrt{1-\rho^2}$, so the gap between points is **1.7**σ at $\rho = 0.5$ but **10.6**σ at $\rho = 0.99$
- At $\rho = 0.99$ the whole bell falls inside today's bin: $P_{33} = 1.00$, so the chain never leaves y_3, although the true process does move

Rouwenhorst in practice: the transition matrix

- You give Rouwenhorst three inputs: persistence ρ, innovation standard deviation σ, and the number of states n_y. It returns a **grid** and a **transition matrix** P
- **Example**: $\rho = 0.9,\ \sigma = 0.1,\ n_y = 3$
- **Step 1**: the two-state matrix, with $p = (1+\rho)/2 = 0.95$:

$$P_2 = \begin{pmatrix} 0.95 & 0.05 \\ 0.05 & 0.95 \end{pmatrix}$$

- **Step 2**: expand it to three states with Rouwenhorst's **recursive rule**:

$$P_3 = \begin{pmatrix} 0.9025 & 0.095 & 0.0025 \\ 0.0475 & 0.905 & 0.0475 \\ 0.0025 & 0.095 & 0.9025 \end{pmatrix}$$

- Read the first row: from low income, the household stays low with probability 90.25%, moves to the middle with 9.5%, and to high income with 0.25%
- For more states, repeat the same expansion

More details

Rouwenhorst in practice: the grid

- **Step 3**: choose the log-income grid. The stationary standard deviation of $\log y$ is

$$\frac{\sigma}{\sqrt{1-\rho^2}} \approx 0.2294$$

Use n_y equally spaced points between $-\sqrt{n_y - 1}\,\sigma/\sqrt{1-\rho^2}$ and $+\sqrt{n_y - 1}\,\sigma/\sqrt{1-\rho^2}$. For three states:

$$\log y \in \{-0.3244,\ 0,\ 0.3244\} \qquad \Longrightarrow \qquad y \approx \{0.723,\ 1,\ 1.383\}$$

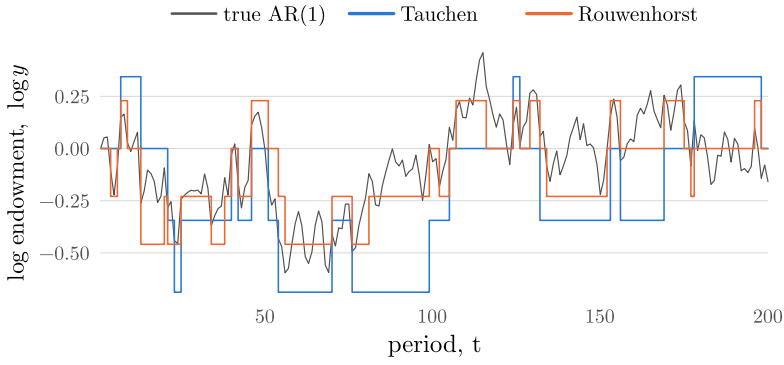
- **Step 4**: use the output in the household problem. From the middle income state:

$$\mathbb{E}\big[V(a', y') \,\big|\, y_2\big] \approx 0.0475\, V(a', y_1) + 0.905\, V(a', y_2) + 0.0475\, V(a', y_3)$$

- The recursive construction preserves the AR(1)'s persistence and moments. Unlike Tauchen, you **never compute areas** under a normal bell curve

Kopecky and Suen 2010

Comparison: Tauchen vs Rouwenhorst



- $n_y = 5$, $\rho = 0.9$, $\sigma = 0.1$. The same shocks ε_t drive all three series: each chain moves to the first state whose cumulative transition probability exceeds $\Phi(\varepsilon_t)$
- With $m = 3$, Tauchen's grid spans ± 0.69; Rouwenhorst's spans $\pm\psi = \pm 0.46$

Discretisation: advanced issues

- **Non-stationary** processes (e.g. life-cycle models): adapt the methods; see Fella, Gallipoli and Pan (2019, RED)
- More general processes (non-normal, asymmetric, correlated shocks): simulation-based methods; see De Nardi, Fella and Paz-Pardo (2020, JEEA)
- **Multivariate** processes: also covered in Tauchen
- Discretisation based on **Gauss–Hermite quadrature** (see Maliar and Maliar)
  - useful if you plan to pre-compute the expectation of the value function, as in Judd, Maliar, Maliar and Tsener (2017, QE)

Household problem

- Once the state space is discretised, the household problem can be solved with a variety of **global methods**:
  - **value function iteration**
  - **policy function iteration**
  - projection methods
- **This week: value function iteration**; policy function iteration follows in week 4
- Strictly speaking, projection is a way to **approximate** the value or policy function, but it is useful to treat it as a separate method

Value function iteration

- With the asset and endowment grids, define the discretised value function on the grid points: $V(a_i, y_j) = V_{ij}$, where V is an $n_a \times n_y$ array
- The goal is to solve the dynamic programming problem:

$$V_{ij} \;=\; \max_{a'_k \geq -\phi} \; \Big\{ u\big(R a_i + w\, y_j - a'_k\big) \;+\; \beta \sum_{m=1}^{n_y} P_{jm}\, V_{km} \Big\}$$

  where the conditional expectation is $\mathbb{E}[V(a'_k, y') \mid y_j] = \sum_{m=1}^{n_y} P_{jm} V_{km}$
- Implementation: the whole continuation array is the matrix product $VP^\top$; compute it once per iteration, not once per grid point

Value function iteration: a 3×3 example

- **Grids**: three asset points and three endowment points ($n_a = n_y = 3$), with no borrowing ($\phi = 0$)
  $a \in \{a_1, a_2, a_3\} = \{0,\ 1,\ 2\}$, $\quad y \in \{y_1, y_2, y_3\} = \{0.723,\ 1,\ 1.383\}$
  - the endowment grid and P are the Rouwenhorst example ($\rho = 0.9$, $\sigma = 0.1$)
- **Markov process**: row j is today's endowment y_j, column m is tomorrow's y_m, and $P_{jm} = \Pr(y' = y_m \mid y = y_j)$

$$V = \begin{array}{c} \\ a_1 \\ a_2 \\ a_3 \end{array} \begin{array}{c} \begin{array}{ccc} y_1 & y_2 & y_3 \end{array} \\ \begin{pmatrix} V_{11} & V_{12} & V_{13} \\ V_{21} & V_{22} & V_{23} \\ V_{31} & V_{32} & V_{33} \end{pmatrix} \end{array} \qquad P = \begin{array}{c} \\ y_1 \\ y_2 \\ y_3 \end{array} \begin{array}{c} \begin{array}{ccc} y_1 & y_2 & y_3 \end{array} \\ \begin{pmatrix} 0.9025 & 0.095 & 0.0025 \\ 0.0475 & 0.905 & 0.0475 \\ 0.0025 & 0.095 & 0.9025 \end{pmatrix} \end{array}$$

- **Value function**: $V_{ij} = V(a_i, y_j)$, the value of a household with assets a_i and endowment y_j: rows are assets, columns are endowments
- **Example**: a household with endowment y_1 today stays at y_1 with probability 0.9025, moves to y_2 with probability 0.095 and to y_3 with probability 0.0025

Value function iteration: a 3×3 example

- **Continuation value**: a household with endowment y_1 today that chooses $a' = a_2$ expects

$$\mathbb{E}[V(a_2, y') \mid y_1] \;=\; P_{11}V_{21} + P_{12}V_{22} + P_{13}V_{23} \;=\; 0.9025\,V_{21} + 0.095\,V_{22} + 0.0025\,V_{23}$$

  the probabilities come from row 1 of P (today's y_1), the values from row 2 of V (the choice a_2)

- **Bellman update** at the state (a_1, y_1): try each of the three choices and keep the best

$$V_{11} \;=\; \max \begin{cases} u(Ra_1 + wy_1 - a_1) + \beta \sum_m P_{1m} V_{1m} & \text{choose } a' = a_1 \\ u(Ra_1 + wy_1 - a_2) + \beta \sum_m P_{1m} V_{2m} & \text{choose } a' = a_2 \\ u(Ra_1 + wy_1 - a_3) + \beta \sum_m P_{1m} V_{3m} & \text{choose } a' = a_3 \end{cases}$$

  blue: today's state, fixed across the three lines; orange: the choice a', which changes from line to line

  repeat for all nine states (a_i, y_j); a choice with $c \leq 0$ gets value $-\infty$

- **All continuation values at once**: entry (k, j) of $VP^\top$ is $\sum_m V_{km} P_{jm} = \mathbb{E}[V(a_k, y') \mid y_j]$
  - one 3×3 matrix product gives all nine continuation values: row k is the choice a'_k, column j is today's y_j

Value function iteration

Under certain conditions the **Banach fixed point theorem** applies (week 2's contraction), and the following iterative procedure converges:

Step 1. Guess an initial value function V_{ij}^n

Step 2. Compute the continuation value using the conditional expectation $\mathbb{E}[V^n(a'_k, y') \mid y_j]$

Step 3. Given the return function and the continuation value, solve the maximisation problem to compute V_{ij}^{n+1} over the whole state space (i, j)

Step 4. Compute the distance $d = \max_{i,j} |V_{ij}^{n+1} - V_{ij}^n|$. If $d <$ tol: done.
Otherwise update $V^n = V^{n+1}$ and repeat

- The **slowest** part of the procedure is solving the maximisation problem

Value function iteration

- Simplest way to solve the maximisation: **brute force grid search**, the method we used **last week** (week 2)

Idea.

- For every (i, j), compute the value for **all** next-period assets a'_k and pick the maximum:

$$V^{n+1}_{ij,k} = \begin{cases} u\left(Ra_i + wy_j - a'_k\right) + \beta\, \mathbb{E}[V^n(a'_k, y') \mid y_j] & \text{if } c_{ij,k} > 0 \\ -\infty & \text{if } c_{ij,k} \leq 0 \end{cases} \qquad V^{n+1}_{ij} = \max_k \left\{ V^{n+1}_{ij,1}, \ldots, V^{n+1}_{ij,n_a} \right\}$$

Issues.

- The optimal policy a' is defined **on-grid**: you need many asset points for a good approximation
- The **curse of dimensionality** bites hard: robust, but can be very slow

Value function iteration

Another way to solve the maximisation: **interpolate** the value function, then use a one-dimensional **optimisation** routine.

Interpolation.

- First interpolate $V^n(a', y')$, then take the conditional expectation
- This gives a **continuous** function of a' (conditional on y): $\mathbb{E}[\hat{V}^n(a', y') \mid y_j] = \hat{V}^n(a'; y_j)$
- Since $u(\cdot)$ is continuous in a', define the continuous objective

$$\varphi(a';\, a_i, y_j) \;=\; u\big(Ra_i + wy_j - a'\big) + \beta\, \hat{V}^n(a';\, y_j)$$

Optimisation.

- Apply a standard optimisation routine to $\varphi(a')$ to find the optimal a'^*
- The value function update is $V_{ij}^{n+1} = \varphi(a'^*;\, a_i, y_j)$

Interpolation: why do we need it?

What we have.

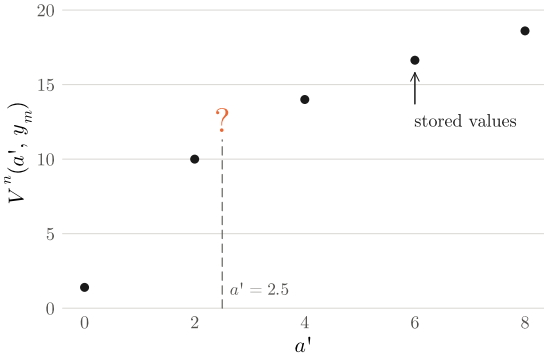
- V^n is a **table**: one number per asset grid point, for each endowment y_m tomorrow
- the figure holds y_m fixed: one column of the table

What we need.

- The household may save **between** grid points, e.g. $a' = 2.5$
- What continuation value goes with that choice?

a'	2	2.5	4
$V^n(a', y_m)$	10	?	14

- **Interpolation**: estimating a function's value between points where its value is known



Linear interpolation: join neighbouring points

- 2.5 is one quarter of the way from 2 to 4, so it gets one quarter of the increase in value:

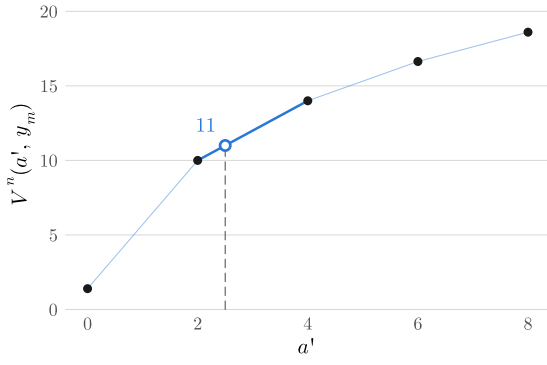
$$\hat{V}^n(2.5, y_m) \;=\; 10 + \underbrace{\frac{2.5-2}{4-2}}_{\text{fraction of the interval}} (14-10) \;=\; 11$$

- In general, for $a_k \le a' \le a_{k+1}$:

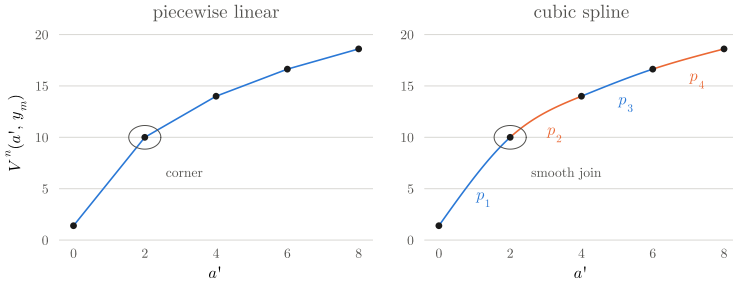
$$\hat{V}^n(a', y_m) \;=\; V^n_{km} + \frac{a'-a_k}{a_{k+1}-a_k}\left(V^n_{k+1,m} - V^n_{km}\right)$$

  the hat marks an interpolated value; V^n_{km} are the stored ones

- **Benefit**: simple, fast and continuous
- **Limitation**: straight segments miss curvature and meet at **corners**
- The line is an **assumption** about the missing values: we have not solved the household problem anywhere along it



Cubic splines: join curved pieces smoothly



- Each interval $[a_k, a_{k+1}]$ gets its own cubic p_k; its four coefficients set its height, slope and bending:

$$p_k(a') \;=\; b_{k0} + b_{k1}(a' - a_k) + b_{k2}(a' - a_k)^2 + b_{k3}(a' - a_k)^3$$

- All coefficients are found **jointly**: each piece passes through its two dots; neighbours share slope and curvature at the joins; two end conditions close the system

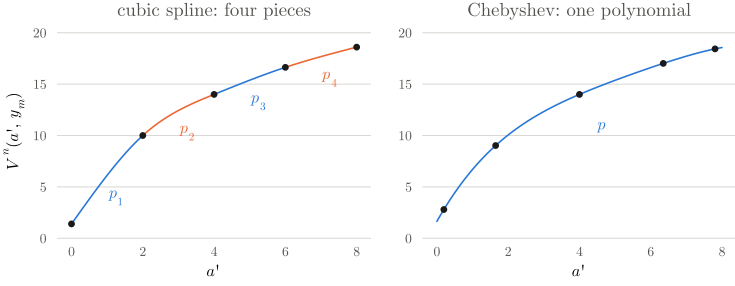
stored values V^n_{km} → solve for the coefficients b_k → evaluate $\hat{V}^n(a', y_m)$ at many trial a'

once per iteration, for each y_m — repeatedly, inside the maximisation

- **Trade-off**: smooth is not the same as accurate: a spline can overshoot and add bends the true function does not have

More details

Chebyshev interpolation: one polynomial across the interval



Construction.

1. Choose **Chebyshev nodes** in $[-\phi, \bar{a}]$ as the asset grid $a_1, \ldots, a_{n_a}$; the stored values V^n_{km} are the values at those nodes
2. Compute the coefficients of the one polynomial through all of them:

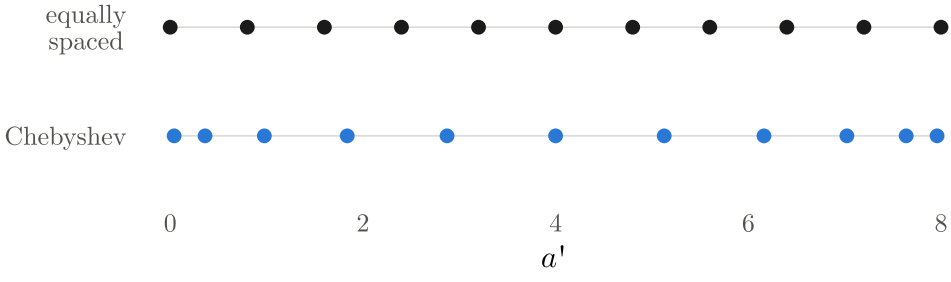
$$\hat{V}^n(a', y_m) \;=\; \sum_{\ell=0}^{n_a-1} \theta^n_\ell(y_m)\, T_\ell\big(x(a')\big)$$

   $x(a')$ rescales $[-\phi, \bar{a}]$ to $[-1, 1]$; T_ℓ are fixed polynomial building blocks (ℓ counts them; k still indexes asset points); $\theta^n_\ell(y_m)$ are recomputed from the current V^n

- **Trade-off**: very accurate for smooth functions; poor at **kinks**, such as the one at the borrowing limit

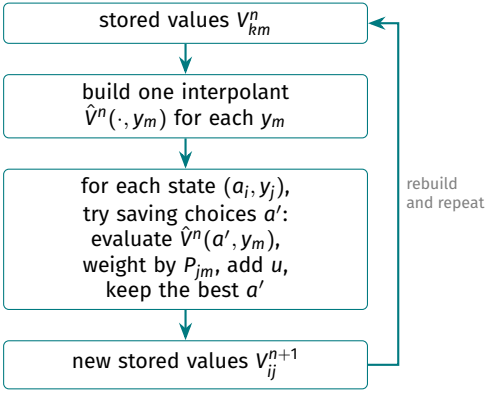
More details

Chebyshev interpolation: where to put the nodes



- Eleven nodes on the same interval: equally spaced (black) vs Chebyshev (blue)
- With equally spaced nodes, a high-degree polynomial can swing wildly near the ends of the interval
- Chebyshev nodes **bunch at the ends** to prevent this; they do not guarantee an accurate fit for every function

Construct once, evaluate many times, then update



- The objective at state (a_i, y_j), now a **continuous** function of a':

$$u\left(Ra_i + wy_j - a'\right) + \beta \sum_m P_{jm}\, \hat{V}^n(a', y_m)$$

  $\hat{V}^n$ supplies the missing value whenever the optimiser tries an a' between grid points

- **One interpolant per** y_m: interpolate over a', one column of V^n at a time
  - one-dimensional, because a' is the only continuous choice
  - today's y_j enters only through the weights P_{jm} (row j of P)
- A one-dimensional **optimisation** routine (golden section, Brent) proposes the trial a'
- V^n changes every iteration, so the interpolants are **rebuilt** every iteration

Interpolation: which method?

Method	Representation	Main strength	Main concern
Linear	straight segments	simple and cheap	corners, missed curvature
Cubic spline	joined cubic pieces	smooth slopes and curvature	overshoot, unwanted bends
Chebyshev	one polynomial over the interval	very accurate for smooth functions	kinks and sharp changes

Optimisation.

- Derivative-free methods (golden section, Brent) work with any interpolant
- Faster Newton-style methods need derivatives, so the interpolant must be differentiable: not linear

Versus grid search.

- Slower per state, but **significantly more accurate** at the same n_a; since fewer grid points suffice, it can end up faster (problem-dependent)

Practical check.

- **Increase the number of grid points** and check whether the saving policy changes materially

Backup: Rouwenhorst's recursive rule

- To go from P_2 to P_3, build four 3×3 matrices, each with P_2 in one corner and zeros elsewhere:
  - $p\,P_2$ in the **top-left**
  - $(1-p)\,P_2$ in the **top-right**
  - $(1-p)\,P_2$ in the **bottom-left**
  - $p\,P_2$ in the **bottom-right**
- Add the four matrices, then divide the **interior** row by two, so every row sums to one:

$$P_3 = \begin{pmatrix} 0.9025 & 0.095 & 0.0025 \\ 0.0475 & 0.905 & 0.0475 \\ 0.0025 & 0.095 & 0.9025 \end{pmatrix}$$

- For n_y states, repeat: build P_n from P_{n-1} in the same way, dividing **all interior rows** by two each time

◂ Back

Kopecky and Suen 2010

Backup: counting the conditions of a cubic spline

- Five grid points give four pieces, each a cubic with four coefficients: $4 \times 4 = 16$ unknowns, so we need 16 conditions



- **The subtlety**: two pieces touch each interior point. At a_2 both must equal the stored value: $p_1(a_2) = V_2$ **and** $p_2(a_2) = V_2$. The first does not imply the second

Requirement	Number
each piece through its two end points	$4 \times 2 = 8$
equal slopes at the joins	3
equal second derivatives at the joins	3
conditions at the two ends	2
total	16

- At each join a_k, $k = 2, 3, 4$:

$$p'_{k-1}(a_k) = p'_k(a_k), \qquad p''_{k-1}(a_k) = p''_k(a_k)$$

- **Natural spline**: $p''_1(a_1) = 0$ and $p''_4(a_5) = 0$, the **second derivative**, not the slope, is zero at the ends

- The same count, starting from the five stored values: three more conditions make neighbouring pieces meet

$$\underbrace{5+3}_{\text{pass through the points and meet}} + \underbrace{3}_{\text{matching slopes}} + \underbrace{3}_{\text{matching second derivatives}} + \underbrace{2}_{\text{end conditions}} = 16$$

◂ Back The linear system

Backup: the natural cubic spline system

- One column y_m at a time; write V_k for V^n_{km}, $h_k = a_{k+1} - a_k$, and M_k for the curvature (second derivative) at a_k
- The curvatures solve one **tridiagonal** linear system, with $M_1 = M_{n_a} = 0$ (natural spline) and, for $k = 2, \ldots, n_a - 1$,

$$h_{k-1}M_{k-1} + 2(h_{k-1} + h_k)M_k + h_k M_{k+1} \;=\; 6\left(\frac{V_{k+1} - V_k}{h_k} - \frac{V_k - V_{k-1}}{h_{k-1}}\right)$$

- The coefficients of piece p_k on $[a_k, a_{k+1}]$ then follow directly:

$$b_{k0} = V_k, \qquad b_{k1} = \frac{V_{k+1} - V_k}{h_k} - \frac{h_k(2M_k + M_{k+1})}{6}, \qquad b_{k2} = \frac{M_k}{2}, \qquad b_{k3} = \frac{M_{k+1} - M_k}{6h_k}$$

- A tridiagonal system costs of order n_a operations: cheap to rebuild every iteration

◂ Back

Judd 1998, ch. 6

Backup: Chebyshev nodes and coefficients

- **Nodes** on $[-1, 1]$, then mapped to the asset interval $[-\phi, \bar{a}]$, for $k = 1, \ldots, n_a$:

$$z_k = -\cos\left(\frac{(2k-1)\pi}{2n_a}\right), \qquad a_k = -\phi + \frac{(z_k+1)(\bar{a}+\phi)}{2}, \qquad x(a') = \frac{2(a'+\phi)}{\bar{a}+\phi} - 1$$

- **Building blocks**: $T_0(x) = 1$, $T_1(x) = x$, and $T_{\ell+1}(x) = 2x\,T_\ell(x) - T_{\ell-1}(x)$
- **Coefficients** for column y_m, with $V_k = V^n_{km}$:

$$\theta^n_0(y_m) = \frac{1}{n_a}\sum_{k=1}^{n_a} V_k, \qquad \theta^n_\ell(y_m) = \frac{2}{n_a}\sum_{k=1}^{n_a} V_k\,T_\ell(z_k), \quad \ell = 1, \ldots, n_a - 1$$

- The resulting polynomial of degree $n_a - 1$ passes through all n_a stored values exactly
- The nodes lie strictly inside $[-\phi, \bar{a}]$: the grid no longer contains the borrowing limit itself

◂ Back

Judd 1998, ch. 6